Agent Workflow

기술 포트폴리오

김준호 / Programmer

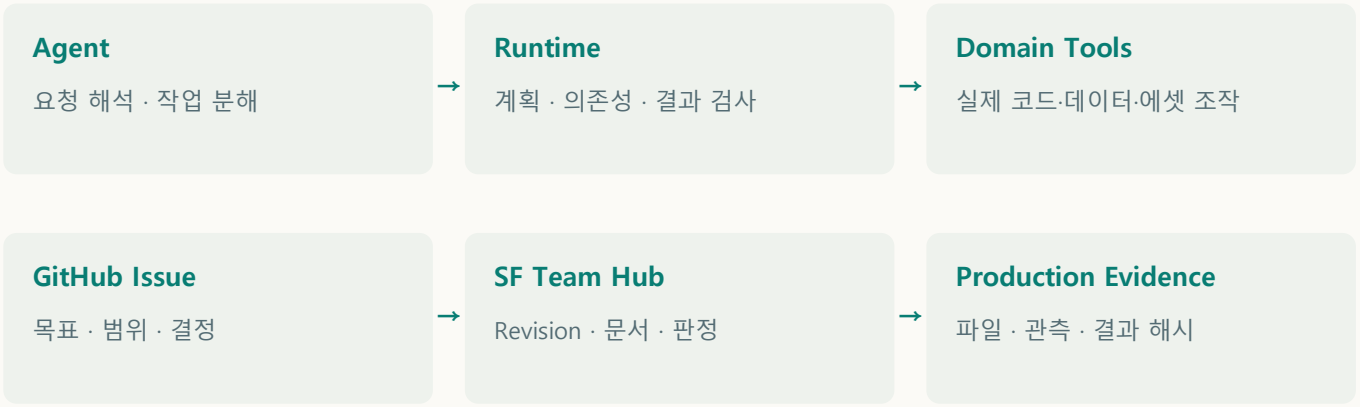
AgentPipeline Runtime
Official Unreal MCP 연동
SF Team Hub 서버 MCP 및 상태 관리

계획 검증 · Tool binding · 요청 중복 처리
버전 충돌 · Job 수명 · Evidence · 저장 후 검증



01 / SYSTEM ARCHITECTURE

Agent·Runtime·실행 도구의 책임 분리



설계 문제

자연어 요청 해석, 실행 가능성 검사, 실제 mutation과 제품 완료 판정이 섞이면 정적 검사 성공이 실제 제작 성공으로 보고될 수 있다.

구현한 경계

Runtime은 선택한 Workflow/Step의 구조와 관측 기록을 검사한다. Agent의 의미 판단과 호스트의 실제 실행은 분리하고, Toolset·배치 도구가 실제 효과를 수행한다.

상태 원본

Issue 작업은 GitHub, Spec 작업은 Hub의 Revision·문서·결정이 원본이다. 실행 결과는 Evidence로 연결하고 대화 기억을 별도 상태 저장소로 사용하지 않는다.

적용 범위

모든 요청에 계획 JSON이나 영속 Issue를 만들지 않는다. 검증된 plan/record를 쓰는 다단계 제작에 Runtime을 적용하고 작은 직접 작업은 현재 요청 범위로 처리한다.

구현 근거 WorkflowRuntime.mjs / spec-model.ts / AGENTS.md

02 / WORKFLOW RUNTIME

계획의 효과·대상·의존성을 검증

Workflow plan은 Goal·요청 Revision과 Step의 intent·targets·expected·effect·dependsOn·requiredCapabilities를 가진다. 허용 효과와 선행 조건을 정적 계약으로 검사한다.

```
// Step contract (field excerpt)
{
  id, intent, targets, expected,
  effect, skills, requiredCapabilities, dependsOn
}

validateWorkflowPlan(root, plan)
resolveWorkflowPlan(root, plan)
```

계획 검사

Step ID 중복·누락, 존재하지 않는 의존성, 순환 의존성을 거절한다. DFS로 그래프를 검사하고 선언한 effect가 plan.allowedEffects에 포함되는지 대조한다.

진행 판정

결과 배열의 순서 대신 의존 그래프 순서로 PASS를 판정한다. 선행 Step이 검증되고 expected별 관측·Evidence가 맞아야 다음 후보로 이어진다.

재검증과 미확정 결과

planHash·requestRevision이 달라지면 REVALIDATION_REQUIRED로 전환한다. RUNNING·UNKNOWN 효과가 남아 있으면 상태 확인을 먼저 요구해 중복 mutation을 피한다.

구현 근거 WorkflowRuntime.mjs: validateWorkflowPlan / evaluateProgress / advanceWorkflow

03 / OFFICIAL MCP

논리 capability와 실제 Toolset 연결

Step 요구
필요한 효과·입출력

→

Binding 후보
Registry · semantic status

→

Live 확인
Toolset · 전체 schema · 조회

설계 문제

Tool 이름이 유사해도 실제 입력·출력·부작용이 요구 효과와 다를 수 있다. 정적 레지스트리에 없는 항목을 기능 부재로 단정하는 것도 오류다.

구현 방식

Registry는 검토한 binding의 색인으로 사용한다. Runtime은 candidateBindings와 pendingDiscovery를 반환하고, 실제 실행 전 Official MCP의 목록·schema·읽기 결과로 효과를 대조한다.

```
requiredCapabilities
  -> registry semanticStatus === "AVAILABLE"
  -> candidateBindings / pendingDiscovery
  -> live discovery + schema/effect validation

resolution.executionAuthorized = false
```

프로젝트 확장

Scene·Asset·Blueprint·DataTable 등 공식 기능을 우선 사용한다. 제작에 반복 필요한 효과는 World·Character·Combat·UI 도메인 Toolset의 최소 확장으로 연결한다.

오류의 의미 분리

DISCOVERY_REQUIRED는 아직 미탐색, BLOCKED는 연결·환경 등 실제 차단, CAPABILITY_GAP은 live 기능과 조합을 대조한 뒤에도 필요한 효과가 없는 상태다.

구현 근거 OfficialCapabilityRegistry.mjs / OfficialCapabilityMap.json / SFEditorToolsets

04 / SERVER MCP

하나의 서버에서 계정·권한·작성자 관리

GPT / Codex
현재 연결된 계정

→

서버 MCP
인증 · scope · 멤버십

→

Hub 명령
버전 검사 · 저장 · 감사

팀원별 플러그인 제작·배포는 채택하지 않았다. 기존 SF Team Hub 서버 MCP를 공통 경로로 사용하고, 허용 범위와 작성자 귀속을 서버에서 확인한다.

인증과 도구 노출

initialize·tools/list와 tools/call을 구분한다. 실제 호출은 grant를 인증하고 scope·멤버 자격을 대조한다. 카탈로그에 보이는 도구가 호출 권한을 대신하지 않는다.

일반 작업과 사람 결정

일반 조회·작성, 제작자 결정, 관리자 명령을 분리한다. 원본 사용자 지시·실제 계정·시각을 결정 이력에 연결하고 AI 결과를 사람 판정으로 저장하지 않는다.

```
MCP tools/call
  -> authenticate(request)
  -> checkGrant(grant)
  -> checkAgentCall(tool, args)
  -> command validation + persist
  -> receipt / readback
```

실제 연결 검증

승인된 계정의 조회·제한 저장·재조회와 새 대화의 Spec 상태 복원을 확인했다. 두 계정 Android 공정은 사용자 확인과 저장 근거를 구분해 기록했다.

구현 근거 lib/remote-mcp.ts / page-tools.ts / agent-api auth·grant 계약

05 / CONCURRENCY & IDEMPOTENCY

낙관적 버전 검사와 요청 중복 처리

동시 편집과 응답 유실을 같은 재시도 문제로 처리하지 않았다. 저장 경쟁은 storeVersion으로, 명령 재전송은 actorId·requestId·inputHash로 검사한다.

```
// persist(): conditional update (simplified)
UPDATE specStore
SET payload = ..., storeVersion = version + 1
WHERE id = specId AND storeVersion = version;

// Same actor + requestId, changed inputHash
throw REQUEST_CONFLICT;
```

저장 경쟁

현재 버전과 일치한 행만 갱신한다. 갱신 행이 없으면 409를 반환하고 현재 문맥을 다시 읽도록 한다. 오래된 전체 payload로 최신 편집을 덮어쓰지 않는다.

동일 명령 재전송

같은 계정·requestId의 기존 명령을 storeVersion 검사보다 먼저 대조한다. 동일 intent는 기존 결과를 사용하고, 같은 ID에 다른 입력은 충돌로 거절한다.

입력 기준 변경

Revision·sourceRevision·문서 버전·설계 버전을 별도로 확인한다. 원본 후보 변경이나 다른 문서 기준의 명령은 현재 Spec에 적용하지 않는다.

구현 근거 spec-store.ts: persist / page-commands.ts: commandReplay·checkCommand

06 / ASYNC JOBS

장시간 작업의 임대·입력·결과 검증

QUEUED
입력 fingerprint · 예약자

→

RUNNING
worker · lease · heartbeat

→

COMPLETED / FAILED
schema · 입력 재대조 · 이력

설계 문제

작업 중 문서·원본·Revision이 바뀌거나 worker가 중단될 수 있다. 실행이 끝났다는 이유만으로 이전 입력의 결과를 현재 상태에 적용하면 stale write가 된다.

임대와 소유권

claim 시 worker와 예약 대상을 대조하고 lease token의 해시·만료 시각을 저장한다. 실행 중 heartbeat와 완료 요청은 현재 lease를 검사한다.

결과 적용 조건

Revision·sourceRevision·active job과 실제 원본을 재확인한다. 입력 fingerprint와 결과 schema·실제 문서 anchor·UML 참조가 맞는 결과만 저장한다.

```
result submission
  -> lease / worker / expiration
  -> revision / source / activeJob / fingerprint
  -> result schema + actual references
  -> persist result and history
```

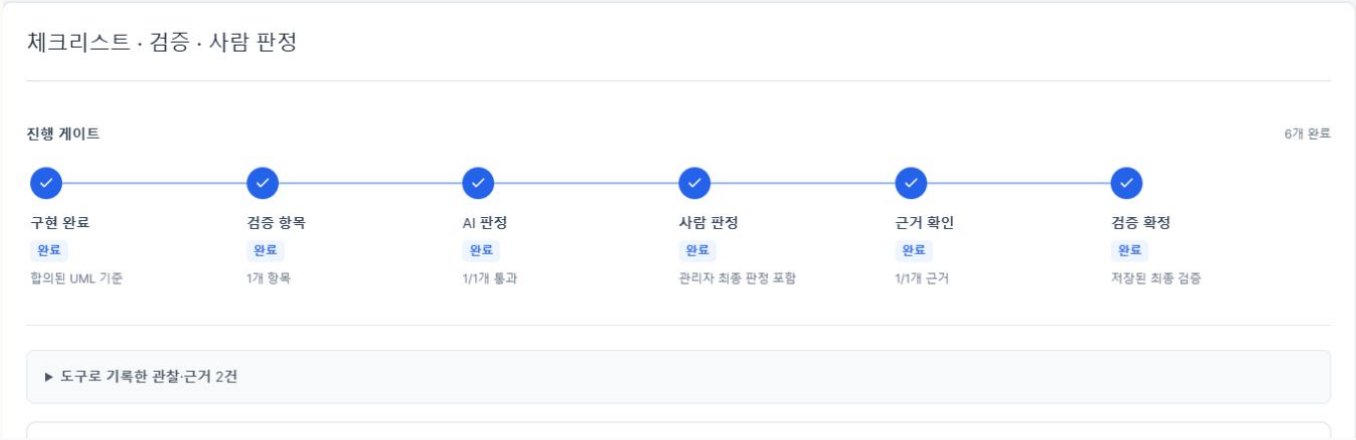
대화와 worker 경로

CURRENT_CHAT 결과는 서버가 내부 인수·완료하고, LOCAL_WORKER는 임대 실행을 사용한다. 요청이 바뀌거나 만료되면 이전 결과를 거절하고 원본·결정 이력을 보존한다.

구현 근거 app/agent-jobs.ts / lib/agent-lease.ts / designJobFingerprint

07 / STATE & EVIDENCE

관찰 기록·구현 완료·판정의 분리



실제 UI는 서버에 저장된 구현·AI·사람·근거·최종 검증 상태를 표시한다.

구현 완료 계약

항목은 현재 설계 버전·재검토 상태를 충족해야 한다. done:true에는 해당 항목의 최신 현재 PASS 기록과 실제 Evidence를 포함하도록 검사한다.

일괄 변경의 원자성

선택한 항목과 기록을 모두 검증한 뒤 done을 변경한다. 다른 항목·과거 Revision의 기록이나 중복 ID를 거절하고, 완료 상태가 바뀌면 lastVerified를 무효화한다.

판정과 발행

관찰 저장, 항목 완료, AI 체크리스트, 사람 수락, VERIFY, 원격 main 확인·CURRENT를 별도 상태로 둔다. 설계·구현 목록 변경의 영향을 받은 판정은 재확인한다.

현재 적용 결과

Spec94의 승인 범위 6개 항목과 근거 정합성 검증을 저장·재조회하고 정식 VERIFY를 확정했다. iOS 보류와 원격 main·발행 대기를 별도로 유지했다.

구현 근거 implementation-sync.ts / verification 계약 / spec-model.ts·spec-store.ts

08 / ENGINE MUTATION & RECOVERY

배치 checkpoint와 실행 결과 재조회

Unreal 제작 호출의 timeout은 실행 실패를 확정하지 못한다. Editor에서 mutation이 진행됐을 수 있으므로 결과가 불명확한 호출은 자동 반복하지 않고 checkpoint와 readback으로 정합성을 확인한다.

preview
대상 · 입력 · 기존 결과

→

write-ahead
running checkpoint 기록

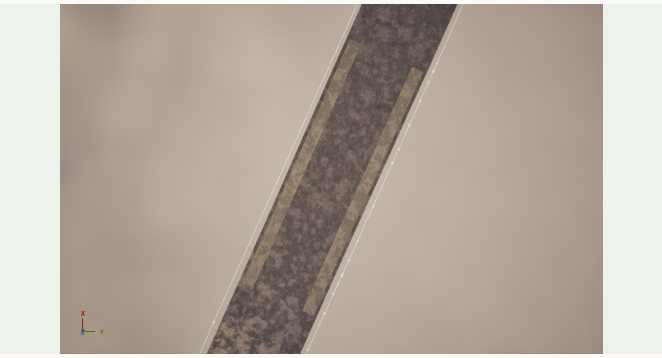
→

apply / readback
실제 저장 · 불명확 상태 분리

구현한 복구 단위

RoadBatch는 batch/plan hash와 청크별 request identity를 checkpoint에 연결한다. 완료·보존·충돌·uncertain을 구분하고, 검증된 기존 청크를 유지한 채 재개한다.





실제 적용 사례 / #54 대표 S001 · 기존 도로면을 보존하며 차선·보도 추가

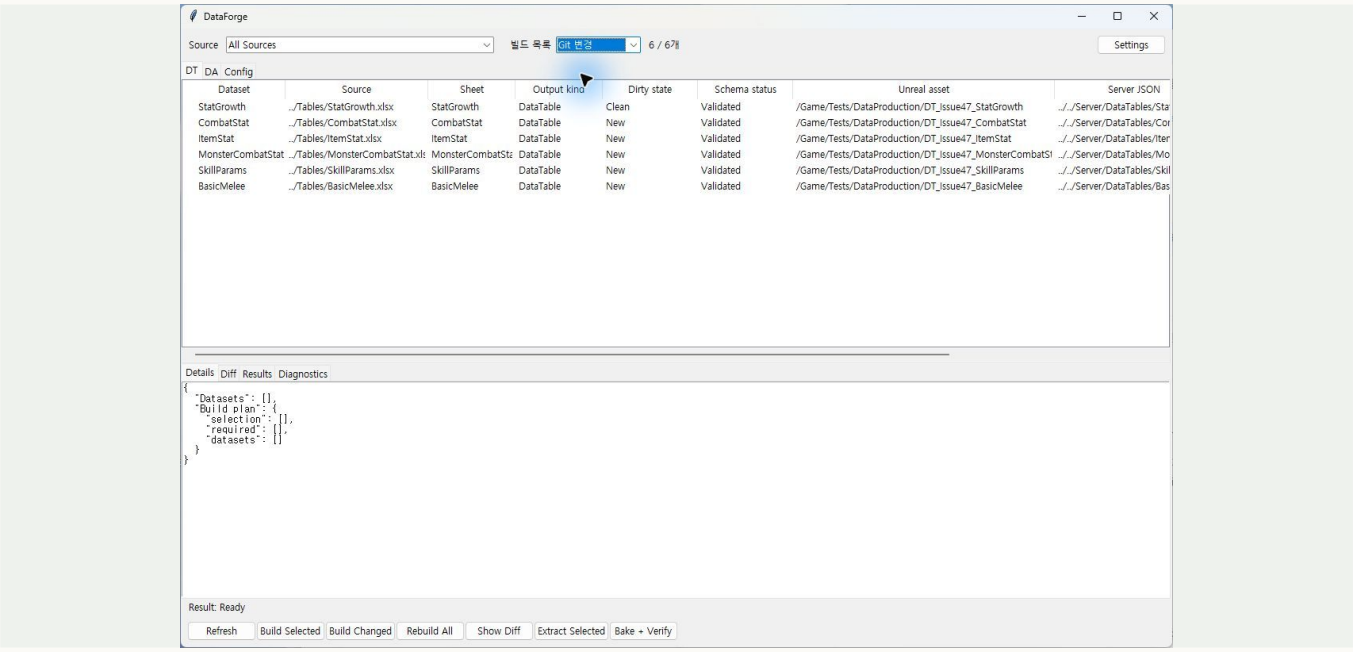
보존과 한계

도메인 소유권·기존 입력·변경 대상과 저장 결과를 대조한다. 전체 World mutation gate의 runner 연동은 문서상 미완료이며, 배치 도구 성공을 전체 월드 보호 보장으로 확대하지 않는다.

구현 근거 RoadBatch.mjs: runBatch / checkpoint·requestHash / Native save readback

09 / PRODUCTION TOOL INTEGRATION

반복 데이터 제작을 공통 Core로 연결



실제 DataForge GUI · GUI·CLI가 같은 Core와 결과 계약을 사용한다.

Authoring source
Excel · Schema · Manifest

→

공통 Core
정규화 · 의미 hash · 검증

→

Engine output
Bake · 독립 Readback · Parity

실행 결과의 검증

정규화한 데이터에서 UE 빌드 입력과 Server JSON을 만들고, 별도 프로세스가 읽은 실제 UDataTable 값과 비교한다. 파일 생성 성공과 값의 일치를 분리했다.

실패·변경 처리

잘못된 입력은 마지막 정상 산출물을 보존한다. Git 변경 후보와 semantic fingerprint를 대조해 의미가 같은 데이터의 불필요한 재생성을 피한다.

DA·Config는 Unreal 원본을 유지하고 native read-only 조회·추출을 사용한다. Workflow가 도메인 도구를 소비하는 실제 제작 사례다.

구현 근거 Tools/DataForge / SFDataBuildCommandlet / 데이터 parity 검증 기록

10 / VALIDATION

정적 계약·서버 회귀·실행 검증의 경계

Runtime 회귀

Step 구조·효과·의존 그래프, 계획 변경, expected와 Evidence 해시, UNKNOWN 결과 이후의 재개 조건을 검사한다. Artifact 경로는 저장소 경계를 벗어나지 않도록 검증한다.

서버 회귀

MCP/OAuth·작성자·scope, 명령 replay·버전 충돌, lease와 stale 결과, 문서 patch·참조, 구현/검증 게이트를 각 책임별 테스트로 분리했다.

실제로 확인한 범위

Spec94 기술 검토에서 MCP/OAuth 22개, rich store 검사, planning-session·agent-lease, 공개 도구 계약과 production build PASS를 기록했다. 전체 TypeScript 검사 기존 오류3건 FAIL도 보존했다.

클라이언트·운영 관측

실제 서버 MCP 저장·readback, 두 승인 계정 Android 사용자 확인, 새 대화 상태 복원을 연결했다. 일부 기록만 읽은 첫 요약의 오류를 전체 필요한 기록 대조로 정정했다.

```
static validation != live tool verification
record saved      != implementation completed
AI verdict        != human acceptance
VERIFY            != remote main / publication
```

검사 숫자는 해당 실행 시점·범위에 한정한다. 이번 PDF 편집에서는 제품 테스트를 새로 실행하지 않았다. 완전 무인 실행·모든 플랫폼 품질·효율 향상률은 주장하지 않는다.

구현 근거 WorkflowRuntime.Tests.mjs / RepositoryAlignmentTest.mjs / Hub tests·Spec94 기록

11 / IMPLEMENTATION SCOPE

구현 범위와 공개 자료

프로그래머 관점의 설계·개발 범위

실행 계약·Runtime 검증, capability binding과 Unreal Toolset 연결, 서버 MCP와 명령·버전·Job·상태 모델, 반복 제작 도구와 Evidence 검증 기반을 구성했다.

규칙과 프로그램의 구분

Workflow·Rule·Skill은 Agent의 판단·절차를 정의한다. Runtime·서버·도메인 Tool은 기계 검증과 실행을 수행한다. 문서가 있다는 사실을 실행기 구현이나 자동 dispatch 완료로 표현하지 않는다.

미완료 범위

Runtime의 host turn 자동 재개·Sub Agent spawn은 구현 범위가 아니다. World mutation gate의 전체 runner 연동, iOS 실측, Spec94 원격 main·발행은 별도 상태로 남아 있다.

UnrealAgentWorkflow 공개 저장소 ↗

구조와 책임 경계 ↗

실제 SF Team Hub 공정 화면 ↗

구현·검증 현황 ↗

원본 대조·공개 범위 기록 ↗

기준일 2026.10.26 · ProjectSF 원본 코드·문서·실행 기록을 대조. 상용 프로젝트의 비공개 코드·자료와 개인 R&D;를 섞지 않았다.